Reflexión Final

Sistema de Mantenimiento Predictivo 4.0

Arquitecturas de Software Distribuidas — Curso 25/26

Gaizka Miranda · Juan María Díaz

1. Introducción

Este documento recoge la reflexión del equipo sobre el proceso de desarrollo del Sistema de Mantenimiento Predictivo 4.0, desarrollado siguiendo un enfoque secuencial inspirado en el modelo en cascada. El sistema consta de dos subsistemas independientes —un backend de Machine Learning (FastAPI + MongoDB) y un frontend web (HTML/JS)— integrados en una fase final.

La reflexión analiza las dificultades encontradas durante las distintas fases del ciclo de vida, compara el enfoque utilizado con la metodología SCRUM y extrae conclusiones sobre qué cambiaría el equipo en una próxima iteración.

2. Problemas de Integración

2.1 Dificultades encontradas

La integración entre el subsistema frontend y el backend ML generó las primeras fricciones técnicas del proyecto. Concretamente, el equipo encontró dos tipos de problemas:

- Desacuerdo en el formato de los datos: el frontend esperaba estructuras JSON que no coincidían exactamente con las que devolvía el backend en un primer momento, lo que requirió renegociar el contrato de la API.
- Ambigüedad en las URLs de los endpoints: al desarrollar los subsistemas de forma independiente, cada subequipo asumió rutas de acceso distintas, lo que ocasionó errores 404 durante las primeras pruebas de integración.

Estos problemas se resolvieron mediante comunicación directa entre los subequipos y el uso de vibe coding asistido por IA, lo que permitió iterar sobre los cambios con rapidez y reducir el tiempo de resolución de conflictos.

2.2 Análisis desde la teoría

> "La integración entre componentes es una de las fases más críticas del ciclo de vida tradicional. Los errores detectados en esta fase tienen un coste de corrección muy superior al de fases anteriores." — Sommerville, 2016

En el enfoque en cascada, la integración ocurre al final del ciclo, lo que hace que los problemas de acoplamiento entre subsistemas se acumulen sin detectarse hasta ese momento. La necesidad de definir y acordar el contrato de la API durante la fase de requisitos o diseño —no durante la integración— habría evitado estos inconvenientes.

La aplicación del patrón de arquitectura cliente-servidor y el uso de CORS en el backend fueron decisiones de diseño correctas que facilitaron la integración, aunque no eliminaron la necesidad de alinear expectativas entre subequipos.

3. Dificultades del Enfoque Tradicional

3.1 La rigidez del modelo en cascada

La principal dificultad del enfoque tradicional fue la imposibilidad de avanzar sin cerrar completamente la fase anterior. En la práctica, el equipo se vio obligado a volver a fases ya completadas cuando surgieron nuevas necesidades durante la implementación:

- Durante la implementación del backend se identificaron requisitos funcionales que no habían sido contemplados en el documento de requisitos (p. ej., el endpoint de estadísticas globales para el dashboard).
- El documento de diseño tuvo que actualizarse para reflejar la incorporación de MongoDB como capa de persistencia, un cambio que surgió durante la implementación y no durante el diseño.

Cada vez que fue necesario retroceder a una fase anterior, se generó un coste adicional en tiempo y documentación, que en un equipo más grande o con plazos más ajustados habría supuesto un riesgo real para la entrega.

3.2 Validación tardía

Otra dificultad inherente al modelo secuencial es que la validación real del sistema ocurre al final. Hasta la fase de pruebas de sistema, el equipo no pudo verificar que ambos subsistemas funcionaban de forma conjunta. Esto contrasta con el principio de entrega continua, donde la integración se realiza de forma frecuente y los errores se detectan de manera temprana.

> "Los requisitos no son estáticos. El entorno del sistema cambia con el tiempo y el sistema debe adaptarse a estos cambios." — UD. 2 SDLC, Análisis de requisitos

Esta característica de los requisitos —su volatilidad natural— choca directamente con la premisa del modelo en cascada, que exige que los requisitos estén completamente cerrados antes de iniciar el diseño.

4. Comparación con SCRUM

4.1 Ventajas de SCRUM para este proyecto

Si el proyecto se hubiera desarrollado con SCRUM, el equipo habría obtenido varias ventajas significativas:

- Adaptación rápida a cambios: en SCRUM, cualquier cambio en los requisitos se incorpora al Product Backlog y puede abordarse en el siguiente sprint. En cascada, un cambio requiere revisar y actualizar formalmente la documentación de fases anteriores, lo que ralentiza el proceso.
- Integración continua desde el inicio: en lugar de integrar los dos subsistemas al final, en SCRUM cada sprint habría producido un incremento integrado y funcional, detectando los problemas de acoplamiento semana a semana en lugar de acumularlos hasta el final.
- Feedback temprano: la revisión de sprint permite al equipo validar el producto con usuarios reales de forma incremental, reduciendo el riesgo de que el sistema final no cumpla las expectativas.
- Gestión del trabajo en equipo de dos personas: con sprints cortos y reuniones de sincronización frecuentes (daily standup), los dos subequipos habrían mantenido una alineación constante sobre el estado del desarrollo, evitando las discrepancias en el contrato de la API.

4.2 Desventajas de SCRUM para este proyecto

Sin embargo, SCRUM también presenta dificultades en el contexto de este proyecto:

- Documentación reducida: en SCRUM se prioriza el software funcional sobre la documentación exhaustiva. Para una entrega académica que exige documentos formales de requisitos y diseño, esto sería un inconveniente.
- Dificultad para planificar la arquitectura completa: la arquitectura de dos subsistemas con un contrato de API bien definido requiere un mínimo de diseño previo. En SCRUM puro, este diseño arquitectónico puede diluirse entre sprints y generar deuda técnica.
- Requiere mayor madurez del equipo: SCRUM exige roles bien definidos (Product Owner, Scrum Master, Development Team) y ceremonias regulares. Para un equipo de dos personas y un proyecto académico acotado, esta sobrecarga puede no estar justificada.

> "Los métodos ágiles suelen utilizar la ingeniería de requisitos incremental y pueden expresar los requisitos en forma de historias de usuario. Esto resulta práctico para los sistemas empresariales, pero es problemático para los sistemas que requieren un análisis previo a la entrega." — UD. 2 SDLC, Análisis de requisitos

5. Qué Cambiaríamos con SCRUM

Si el equipo afrontara de nuevo este proyecto con SCRUM, los principales cambios serían:

- Definir el contrato de la API (endpoints, formatos JSON, códigos de respuesta) en el primer sprint como parte del backlog de integración, antes de que cada subequipo desarrolle su subsistema de forma independiente.
- Establecer sprints de dos semanas con una revisión conjunta al final de cada sprint, donde ambos subequipos integren sus desarrollos parciales y validen el funcionamiento conjunto.
- Gestionar el backlog con historias de usuario priorizadas en lugar de documentos de requisitos cerrados, lo que permitiría incorporar nuevas funcionalidades (como el endpoint de estadísticas) sin necesidad de revisar formalmente toda la documentación.
- Introducir pruebas automatizadas desde el primer sprint, haciendo de la cobertura de tests un criterio de Definition of Done, en lugar de añadir pruebas al final del ciclo.
- Usar el tablero Kanban o Scrum Board para visualizar el estado de cada subsistema en tiempo real y detectar bloqueos entre los dos subequipos de forma temprana.

En definitiva, SCRUM habría reducido el coste de los cambios tardíos y mejorado la comunicación entre subequipos, a cambio de una menor rigurosidad documental y una mayor exigencia en la coordinación continua.

6. Fase más Problemática

6.1 Control de versiones con Git

La gestión de ramas en Git fue uno de los mayores retos del proyecto. Al inicio, el repositorio carecía de la estructura de ramas recomendada (Gitflow: main → develop → feature/*), lo que generó confusión sobre dónde integrar los cambios de cada subequipo y dificultó la trazabilidad del historial.

La estructura correcta —crear una rama develop como punto de integración intermedia entre las ramas feature y la rama main— es fundamental en proyectos con varios subequipos, ya que permite aislar el desarrollo de cada subsistema sin afectar directamente a la versión estable. Según la teoría del curso:

> "Main: almacena el historial de publicación oficial. Develop: rama de desarrollo que sirve como rama de integración para las funciones. Feature: rama de nueva funcionalidad, a partir de develop." — UD. 2 SDLC, Despliegue

Este aprendizaje fue especialmente valioso: en proyectos reales, una mala gestión de ramas puede provocar pérdida de trabajo, conflictos de merge difíciles de resolver y dificultades para auditar el historial de cambios.

6.2 Pruebas automáticas

La fase de pruebas también resultó exigente, especialmente a la hora de diseñar pruebas que no dependieran de la infraestructura real (MongoDB, modelo ML) para poder ejecutarse en el pipeline de CI/CD. El equipo tuvo que implementar objetos fake (FakeCollection, FakeDB) para simular la base de datos en memoria, y mocks del modelo de Machine Learning para que los tests fueran reproducibles y rápidos.

La distinción entre tipos de pruebas —unitarias, de componente y de sistema— no siempre fue evidente al inicio:

- Pruebas unitarias: verifican funciones aisladas (p. ej., clasificar_estado()).
- Pruebas de componente: verifican un endpoint completo con su lógica interna pero con dependencias mockeadas.
- Pruebas de sistema: verifican flujos end-to-end que involucran múltiples componentes integrados, comprobando la consistencia de datos entre endpoints y el contrato de la API con el frontend.

El esfuerzo invertido en las pruebas, aunque costoso en tiempo, demostró su valor al detectar errores de integración antes de que llegaran a producción y al garantizar que el pipeline de CI/CD en GitHub Actions tuviera un criterio objetivo de calidad mediante SonarCloud.

7. Conclusión

El desarrollo de este proyecto bajo un enfoque tradicional mixto ha permitido al equipo experimentar de primera mano las fortalezas y limitaciones del modelo en cascada. La obligación de cerrar cada fase

antes de avanzar impuso una disciplina documental que, en un contexto real con requisitos cambiantes, habría supuesto un coste significativo.

Las fases más costosas —control de versiones y pruebas— son precisamente las que más se beneficiarían de una cultura DevOps y de un enfoque ágil, donde la integración continua y las pruebas automáticas forman parte del flujo normal de trabajo desde el primer día, y no son tareas que se afrontan al final del ciclo.

La comparación con SCRUM revela que, para proyectos con requisitos volátiles y equipos pequeños, la entrega incremental y la retroalimentación continua son ventajas difíciles de ignorar. Sin embargo, el enfoque tradicional ha sido útil para estructurar el pensamiento, definir una arquitectura sólida desde el inicio y producir documentación que sirva de referencia durante todo el ciclo de vida del sistema.